

WAD Demo — Presenter Runbook

Numbered script. Each step shows what to **Say**, **Type**, and **Show**. Steps 5, 6, 7 are **real machine only** — they will not work on the legacy/local dry-run.

Quick Start — Run the demo now

Use this when the machine is already set up and you just want to present.

1. Load the pinned sbx (every new terminal):

```
cd /Users/ajeetraina/work/wad26/wad-demo-presenter
. runtime/.install-state/env.sh
```

2. Confirm it's ready:

```
sbx ls
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:3001/
```

Expected: wad-coding-factory shows running, and 200.

3. Open the app in the browser:

```
open http://127.0.0.1:3001/
```

4. Open the coding agent (present from this terminal):

```
./demo agent
```

5. Inside the Claude prompt, run the network demo:

```
! curl https://github.com/
! curl https://example.org/
```

GitHub responds; example.org returns a 403 policy denial.

6. Optional — show the kit and environment files (second terminal):

```
cat runtime/kits/demo-node/spec.yaml
cat runtime/sbxenv.yaml
```

Recovery — if the sandbox is missing or the app returns 000

Phases are already "done", so this is a repair (not a first-time setup):

```
cd /Users/ajeetraina/work/wad26/wad-demo-presenter
. runtime/.install-state/env.sh
./demo repair --yes
```

Fallback if repair fails: `./demo setup --local-only`. First-time-ever install instead: `sh __INSTALL_ME.sh 3` from the wad26 folder.

Part A — Before you start (once per session)

1. Open two terminal tabs in the presenter folder:

```
cd /Users/ajeetraina/work/wad26/wad-demo-presenter
. runtime/.install-state/env.sh
sbx version
```

sbx version should read v0.45.0-rc2

2. Open browser tabs: <http://127.0.0.1:3001/> (app), governance console, cloud console.
3. **Say:** "Docker is becoming an AI company: give coding agents room to work, make it repeatable, and manage access."

Part B — The demo (in order)

1. Autonomy needs isolation

1. Type (terminal 1) — open the coding agent (brings the sandbox up if needed):

```
./demo agent
```

Raw equivalent, only if the sandbox already exists: `sbx run --name wad-coding-factory`

2. Type at the Claude prompt:

```
Describe the project for me.
```

3. **Say:** "Claude runs inside a local microVM. It can build and serve the app, run containers for tests, and update its own tools — all isolated from my laptop."

2. Reusable kits and environments

1. Type (terminal 2):

```
cat runtime/kits/demo-node/spec.yaml
cat runtime/sbxenv.yaml
```

2. **Say:** "Kits are reusable packages of tools, config and agent instructions. This Node kit supplies package tooling and declares registry access. The environment file ties kits, workspace, resources and ports together — like Compose. These live next to the project, so the next developer gets the same setup."

3. A working application

1. Show: open <http://127.0.0.1:3001/> (Incident Review — AI failure field notes)

2. **Say:** "This app runs inside the sandbox, reachable through a published port. The agent has its own clone of the project; we choose which host directories it can access."

4. Network and credentials (the key moment)

1. Type at the Claude prompt (! runs a shell command inside the sandbox):

```
! curl https://github.com/  
! curl https://example.org/
```

2. Show: GitHub returns a response; example.org returns a policy denial (403, "Denied by local rule").
3. **Say:** "A proxy outside the agent controls network access. GitHub is allowed; example.org is denied by this sandbox's local rule. The same proxy supplies model auth — the agent never sees the API key."
4. Optional contrast: run `curl https://example.org/` on the host (returns 200) to prove the policy only applies inside the sandbox.

5. Organization governance (real machine only)

1. Show in the governance browser: network policy, MCP tool allow/deny, and one earlier audit row.
2. **Say:** "Teams manage access centrally with Docker AI Governance. Here's the org network policy. MCP permissions can allow one tool and deny another through the Sandboxes gateway. The audit log records the resource, decision and time — here's an earlier denial for the same destination."

6. Cloud Sandboxes (real machine only)

1. Type (terminal 2):

```
sbx --cloud ls  
sbx --cloud attach <prepared-sandbox-id>
```

Match the sticker: 03 uses wad-demo-03-...

2. **Say:** "Adding --cloud runs the sandbox on Docker's infrastructure. Here's one already running — I connect from my terminal and come back to the agent's conversation. The session stays in the cloud when I disconnect."

7. Pick up the work (real machine only)

1. Type at the cloud Claude prompt:

```
What did you change, and how did the tests go?
```

2. Type to show tests again:

```
! cd /home/agent/wad-cloud-demo && python3 -m unittest -v
```

3. **Say:** "We gave this agent a small task earlier. The change and passing tests are here, ready for the next instruction — longer tasks keep running while I'm away."

8. DHI / Scout supply-chain (works on this machine)

1. Type — open the security sandbox shell:

```
sbx exec -it wad-dhi-scout bash
```

2. Type inside that shell:

```
export
DHI_IMAGE=dhi.io/node@sha256:61b89b9bd1551723b56de57bf41cc62daefffb266d80636a83cf6b85e35bf17d1
docker run --rm --network none "$DHI_IMAGE" node --version
docker scout quickview "$DHI_IMAGE"
docker scout cves --only-severity critical,high "$DHI_IMAGE"
```

3. **Say:** "The agent has its own Docker Engine. We start from a Docker Hardened Image, and Scout shows packages, vulnerabilities and attestations — information to act on before shipping. The kit tells the agent to run these checks as part of its normal workflow."

Part C — Reset (between visitors)

```
cd /Users/ajeetraina/work/wad26/wad-demo-presenter
./demo reset
```

Verify anytime

```
cd /Users/ajeetraina/work/wad26/wad-demo-presenter
. runtime/.install-state/env.sh
./demo check --model
```

This machine (legacy dry-run): steps 1–4, 8 and reset work. Steps 5–7 need the provisioned demo/org accounts.

WAD Demo — Commands Cheat Sheet

Commands only, top to bottom. Steps 5–7 need the real provisioned machine.

Setup (per session)

```
cd /Users/ajeetraina/work/wad26/wad-demo-presenter
. runtime/.install-state/env.sh
sbx version
```

1. Isolation

```
sbx run --name wad-coding-factory
```

At Claude prompt: Describe the project for me.

2. Kits and environments

```
cat runtime/kits/demo-node/spec.yaml
cat runtime/sbxenv.yaml
```

3. Working app

```
open http://127.0.0.1:3001/
```

4. Network and credentials

At Claude prompt:

```
! curl https://github.com/
! curl https://example.org/
```

Optional host contrast:

```
curl https://example.org/
```

5. Governance (real machine only)

Browser: network policy, MCP allow/deny, one audit row.

6. Cloud sandboxes (real machine only)

```
sbx --cloud ls
sbx --cloud attach <prepared-sandbox-id>
```

7. Pick up the work (real machine only)

At cloud Claude prompt:

```
What did you change, and how did the tests go?
```

```
! cd /home/agent/wad-cloud-demo && python3 -m unittest -v
```

8. DHI / Scout

```
sbx exec -it wad-dhi-scout bash
export
DHI_IMAGE=dhi.io/node@sha256:61b89b9bd1551723b56de57bf41cc62daeffb266d80636a83cf6b85e35bf17d1
docker run --rm --network none "$DHI_IMAGE" node --version
docker scout quickview "$DHI_IMAGE"
docker scout cves --only-severity critical,high "$DHI_IMAGE"
```

Reset

```
cd /Users/ajeetraina/work/wad26/wad-demo-presenter
./demo reset
```

Verify

```
cd /Users/ajeetraina/work/wad26/wad-demo-presenter
. runtime/.install-state/env.sh
./demo check --model
```